Taller Práctico: De Cero a API REST con Django (Flujo Completo)

Objetivo de la Sesión: Configurar nuestro entorno de trabajo en Python, construir nuestro primer modelo de base de datos utilizando el ORM de Django y exponer esos datos a través de una API REST funcional utilizando Django REST Framework (DRF).

Fecha: 17 mar 2026 10:00

Paso 1: Inicialización y Entorno Virtual

En la ingeniería backend con Python, nunca instalamos dependencias globalmente. Siempre aislamos nuestros proyectos.

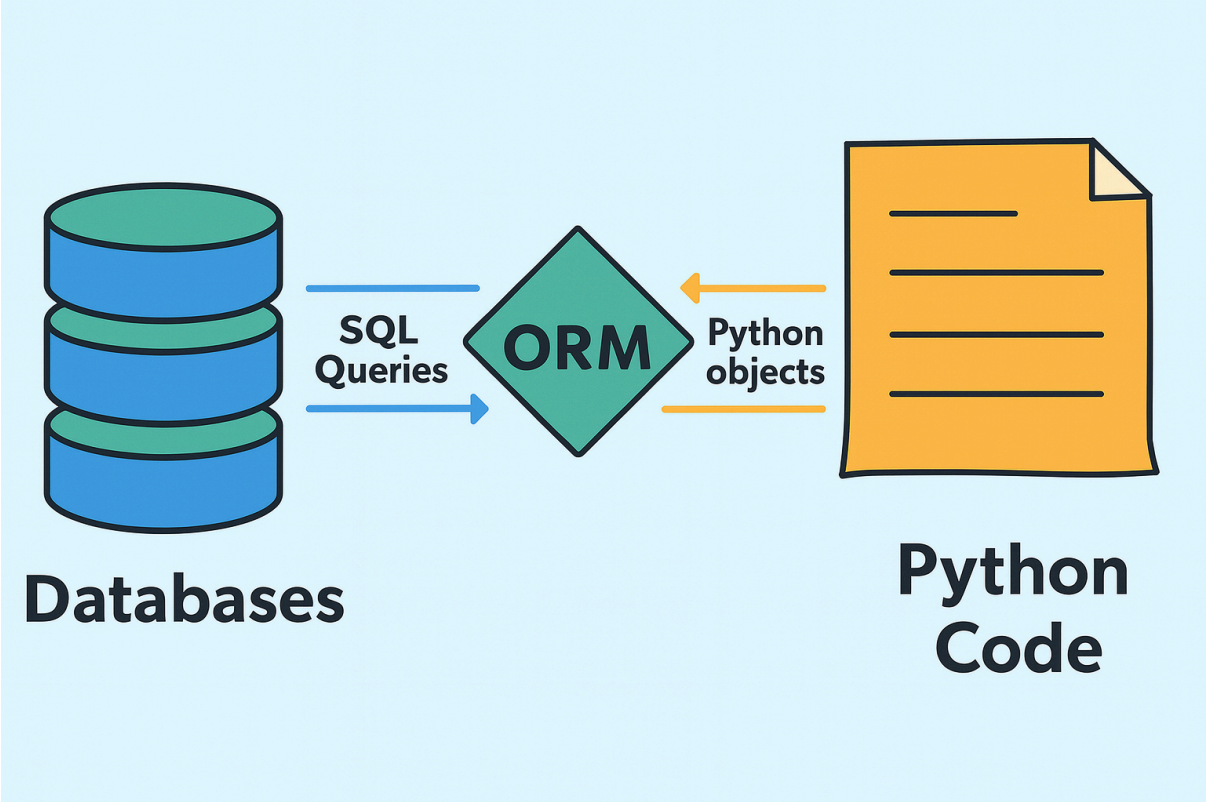
1. Abran su terminal en la carpeta donde vivirá su proyecto (ej. `backend-clinica`).
2. Creen y activen el entorno virtual:
   - **Windows:** `python -m venv venv` y luego `venv\Scripts\activate`
   - **Mac/Linux:** `python3 -m venv venv` y luego `source venv/bin/activate`
3. Instalen el núcleo de nuestra arquitectura:
   - `pip install django djangorestframework`
4. Creen el proyecto principal y su primera aplicación:
   - `django-admin startproject core .` *(No olviden el punto al final)*
   - `python manage.py startapp api`

Configuración vital: Abran `core/settings.py` y registren sus aplicaciones en `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    # ... apps por defecto de django ...
    'rest_framework',
    'api',
]
```

Paso 2: El Modelo de Datos (ORM)

Vamos a traducir una entidad del mundo real a una tabla de base de datos sin escribir SQL, utilizando clases de Python.



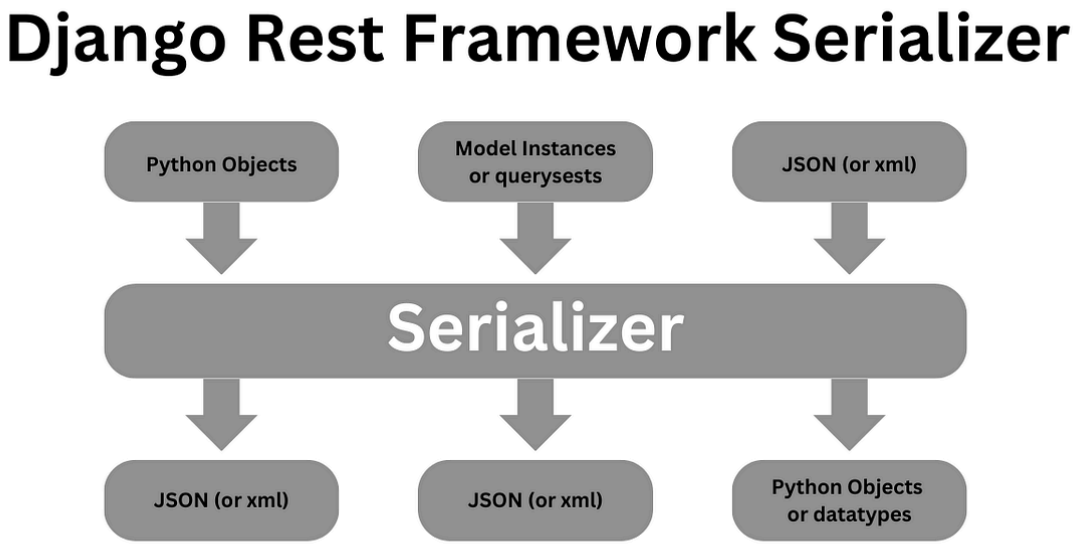
1. Abran `api/models.py` y definan su primer modelo (usaremos `Paciente` como ejemplo):

```
from django.db import models

class Paciente(models.Model):
    nombre = models.CharField(max_length=100)
    apellidos = models.CharField(max_length=100)
    telefono = models.CharField(max_length=15)
    fecha_registro =
models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.nombre} {self.apellidos}"
```

2. **Migraciones:** Díganle a Django que prepare las instrucciones SQL y las ejecute en la base de datos SQLite (por defecto):
    - `python manage.py makemigrations`
    - `python manage.py migrate`

Paso 3: El Traductor (Serializers)

El ORM nos da objetos de Python, pero React necesita JSON. El Serializador hace esta traducción automáticamente.

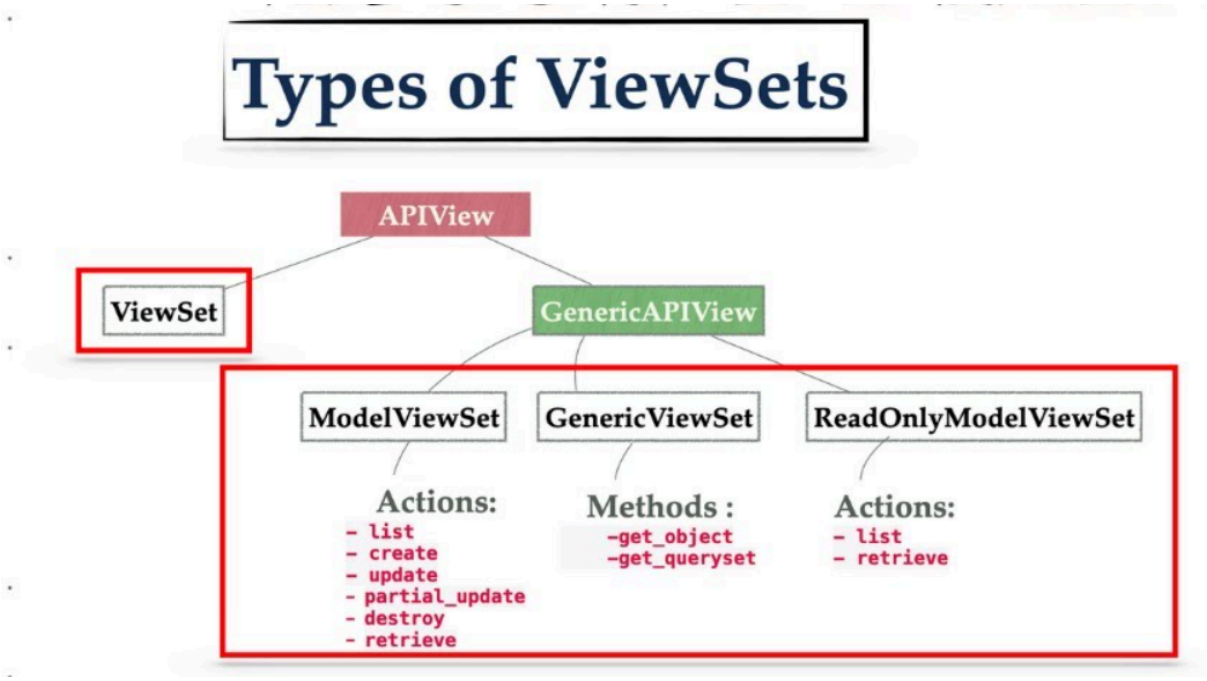


1. Creen un nuevo archivo llamado `serializers.py` dentro de la carpeta `api`.

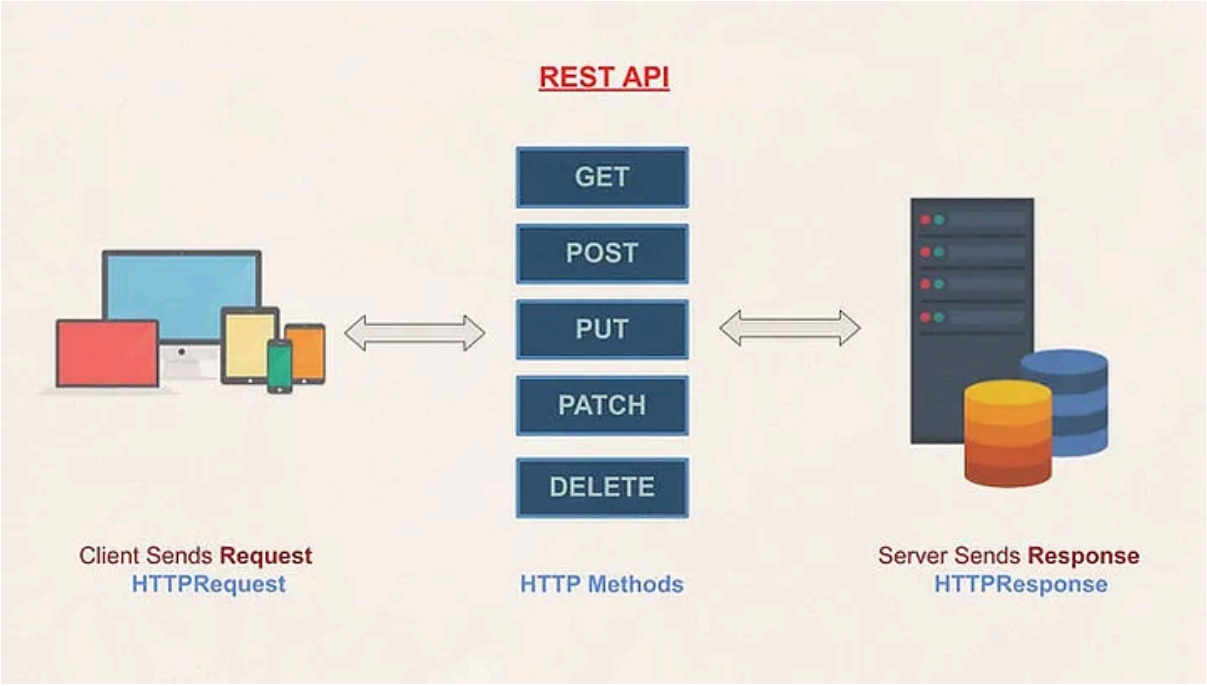
Escriban el siguiente código:

```
from rest_framework import serializers
from .models import Paciente

class PacienteSerializer(serializers.ModelSerializer):
    class Meta:
        model = Paciente
        fields = '__all__' # Expone todos los campos
        read_only_fields = ('fecha_registro',) # Protege
la fecha para que no sea modificada
```

Paso 4: El Controlador (ViewSets)



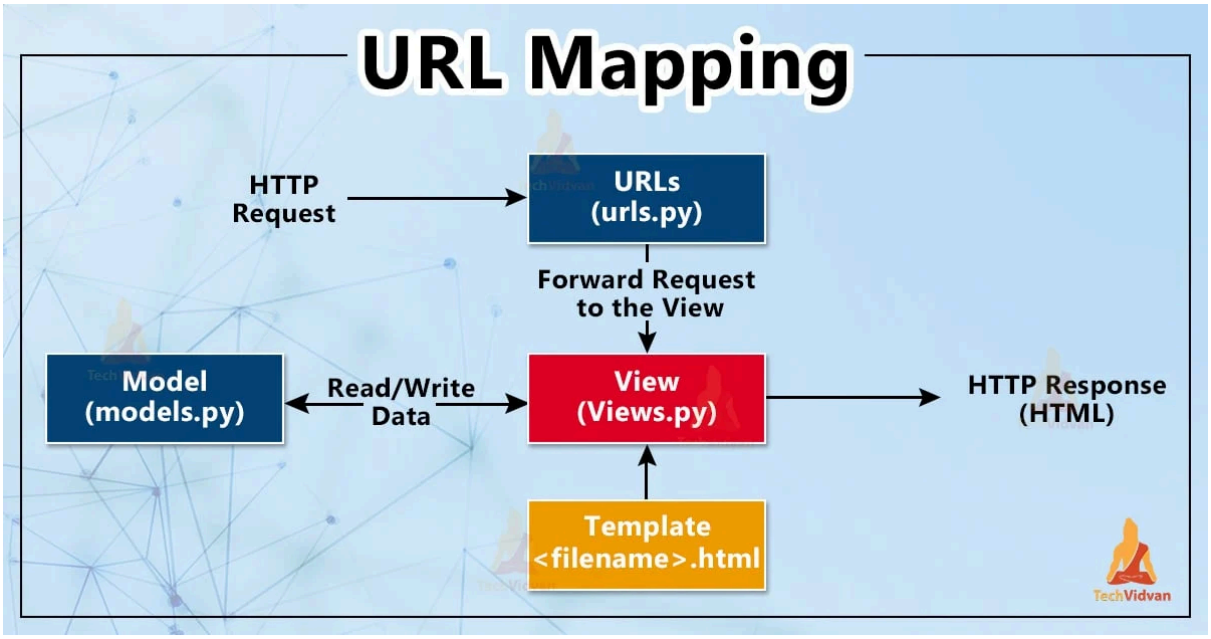
Aquí es donde DRF brilla. En lugar de programar a mano el GET, POST, PUT y DELETE, usaremos un `ModelViewSet` que abstrae toda esa lógica en 3 líneas.



Abran `api/views.py`:

```
from rest_framework import viewsets
from .models import Paciente
from .serializers import PacienteSerializer

class PacienteViewSet(viewsets.ModelViewSet):
    queryset = Paciente.objects.all()
    serializer_class = PacienteSerializer
```

Paso 5: El Enrutador (Routers y URLs)



Finalmente, conectamos nuestra vista a una URL de internet.

1. Creen un archivo `urls.py` dentro de la carpeta `api` y configuren el enrutador:

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import PacienteViewSet

router = DefaultRouter()
router.register(r'pacientes', PacienteViewSet,
basename='paciente')
```

```
urlpatterns = [
    path('', include(router.urls)),
]
```

2. Conexión Principal: Abran el archivo `urls.py` principal (el que está en la carpeta `core`) e incluyan las rutas de su app:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('api.urls')), # Conecta con
nuestra API
]
```

Paso 6: Ejecución y Pruebas del CRUD Completo (Thunder Client / Postman)

¡Momento de la verdad! Al utilizar `ModelViewSet`, DRF les ha generado automáticamente todas las operaciones CRUD. Deben probar cada una de ellas para asegurar la calidad de su API.

1. Levanten el servidor: `python manage.py runserver`
2. Abran su cliente REST ([Thunder Client en VS Code]() o Postman).

Necesitaremos una herramienta para simular las peticiones que en el futuro hará nuestro Frontend (React). Utilizaremos **Thunder Client**, una extensión ultra ligera que vive dentro de nuestro propio Visual Studio Code.

Si nunca han utilizado un cliente REST o no saben cómo hacer peticiones GET, POST, PUT y DELETE, es **obligatorio** que vean este tutorial de 10 minutos:

📺 **[Ver Tutorial de Thunder Client aquí]**
https://www.youtube.com/watch?v=BAk2hQgqLDk

Presten especial atención al minuto 05:50, donde se explica cómo enviar datos en formato JSON a través del método POST. ¡Esa será la clave para aprobar el entregable de hoy!

3. Realicen y documenten las siguientes 5 peticiones apuntando a `http://localhost:8000/api/pacientes/`:
    - **1. CREATE (POST):** Envíen un cuerpo JSON con los datos de un paciente. *Deberían recibir un código `201 Created`.*
    - **2. READ LIST (GET):** Hagan una petición a la ruta raíz `/api/pacientes/` para ver la lista con el paciente recién creado. *Código `200 OK`.*
    - **3. READ DETAIL (GET):** Hagan una petición apuntando al ID específico (ej. `/api/pacientes/1/`). *Código `200 OK`.*
    - **4. UPDATE (PUT/PATCH):** Actualicen el teléfono o apellido del paciente enviando un JSON a la ruta del ID (`/api/pacientes/1/`). *Código `200 OK`.*
    - **5. DELETE:** Envíen una petición DELETE a la ruta del ID (`/api/pacientes/1/`) para eliminar el registro. *Deberían recibir un código `204 No Content`.*

Reto de Laboratorio (Uso de NotebookLM)

Ahora que comprenden el flujo completo, abran su *Product Backlog*. Tomen la primera Historia de Usuario de su proyecto final y utilicen a su "Arquitecto Asistente" (NotebookLM) con el siguiente *prompt*:

> *"Actúa como mi Ingeniero Backend Senior. En el taller de hoy aprendí el flujo de Modelos, Serializadores y ViewSets usando DRF. Basándote en esta Historia de Usuario de mi proyecto: **[Pegar Historia de Usuario]**, genérame el código exacto para el `models.py`, `serializers.py` y `views.py` que necesito para implementar esta funcionalidad."*

Entregable del día:

1. Repositorio en GitHub actualizado con su propio modelo, serializador, vista y urls funcionando.

2. Capturas de pantalla (o exportación de la colección de Thunder Client/Postman) demostrando que ejecutaron exitosamente las **5 operaciones del CRUD** sobre su propia entidad, mostrando claramente los códigos de estado HTTP (200, 201, 204) devueltos por el servidor.